Quadratic-Form Engine / K-COS

A Lean Companion for Exact Finite Algebra and Error-Budget Composition

Komorebi P.

Final edition, 7 October 2026

This companion supplies a focused formal layer for the traditional manuscript. The source formalizes exact finite Gram algebra, refinement invariance, streamed sample-kernel actions, positivity, coefficientwise order, and arithmetic consequences of valid error bounds. It makes the hypotheses behind these claims visible and provides reproducible compiler evidence.

The mathematical engine is larger than this formal layer. Gaussian reduction, Fourier inversion, exponential tilting, COS support and omitted-mode bounds, eigenvalue inclusion, and the interval backend remain outside these modules. An accepted exact-real Lean statement is distinct from a validated numerical implementation. A statement whose hypotheses include an enclosure does not establish how that enclosure was obtained.

Fresh formal verification: passed. The project contains 26 checked theorem declarations and 8 definitions. The pinned Lean 4.34.0-rc2 compiler accepted the project build, both mathematical source modules, the root import, and the 34-declaration axiom audit. Every audited project declaration uses only `propext`, `Classical.choice`, and `Quot.sound`; no `sorryAx` or custom axiom occurs. A valid arithmetic control was accepted without axioms. A false arithmetic control and an acceptance theorem missing its error-budget premise were both rejected with exit code 1. This certifies the finite algebra and arithmetic claims listed in this companion, under their stated hypotheses.

Contents

1 How the formal model represents the engine . . . 3
2 Exact block refinement . . . 3
3 Streamed actions and the sum-of-squares identity . . . 4
4 A complete exact algebra example . . . 4
5 Supplied coefficient intervals . . . 5
6 Enclosures and error ledgers . . . 5
7 Safe transformations of an enclosure . . . 6
8 Declaration inventory and manuscript concordance . . . 6
9 Reproduction and trusted dependencies . . . 7
10 Remaining formal obligations . . . 7

1 How the formal model represents the engine

The factor has a finite sample index type I and an arbitrary variant index type V. A variant block is a finite set $s \subseteq V$. The factor entries are exact real numbers, $A : I \times V \to \mathbb{R}$. The Lean type is `Row->Variant->Real`. The sample coordinates carry a `Fintype` instance. The variants carry a `DecidableEq` instance to support finite-set unions, singleton blocks, and reindexing.

The residualized weighted factor in the traditional paper is

$$A = M_X G W^{1/2}.$$

For this companion, A is supplied. Proving that a particular floating-point residualization procedure computes that mathematical factor would require additional implementation and numerical-error reasoning. These modules do not assume a covariance model or a distribution for the vector v.

There are five central finite-algebra definitions:

$$\begin{aligned}
\text{gramEntry}(s, A, i, j) &= \sum_{k \in s} A_{ik} A_{jk}, \\
\text{matrixAction}(K, v, i) &= \sum_{j \in I} K_{ij} v_j, \\
\text{streamedAction}(s, A, v, i) &= \sum_{k \in s} A_{ik} \sum_{j \in I} A_{jk} v_j, \\
\text{quadraticForm}(K, v) &= \sum_{i \in I} v_i \, \text{matrixAction}(K, v, i), \\
\text{factorEnergy}(s, A, v) &= \sum_{k \in s} \left(\sum_{i \in I} A_{ik} v_i \right)^2.
\end{aligned}$$

The source uses ordinary finite sums in $\mathbb{R}$. The factor convention is the sample Gram $AA^\top$, matching `stream_sample_gram`; it is not the dual Gram $A^\top A$. The paper's statistic $Q = \frac{1}{2} v^\top K v$ requires the additional scalar factor $\frac{1}{2}$ when applying these identities.

2 Exact block refinement

The principal refinement theorem is `gramEntry_union`. Its hypotheses are finite sets $s, t \subseteq V$ and $\text{Disjoint}(s, t)$. It proves, for all sample coordinates i, j,

$$\text{gramEntry}(s \cup t, A, i, j) = \text{gramEntry}(s, A, i, j) + \text{gramEntry}(t, A, i, j).$$

Disjointness prevents repeated columns from being counted twice. It is a mathematical condition, not a performance assumption. The theorem applies to empty blocks and zero factors. It does not require positive weights, since A is already supplied as a real factor.

The proof unfolds `gramEntry` and invokes the finite-sum union identity. No numerical approximation is involved. Repeated application establishes invariance along any explicitly checked sequence of binary disjoint splits. The source does not encode a whole scheduler or its memory allocation policy. A scheduler correctness proof would additionally track complete coverage, disjointness, and termination at every split.

The same refinement has formal consequences for the operator action and the quadratic statistic:

$$\begin{aligned}
(K_{s \cup t} v)_i &= (K_s v)_i + (K_t v)_i, \\
v^\top K_{s \cup t} v &= v^\top K_s v + v^\top K_t v.
\end{aligned}$$

These are `matrixAction_gram_union` and `quadraticForm_gram_union`. The independent `streamedAction_union` proves the corresponding streamed accumulation identity directly.

The reindexing theorem `gramEntry_reindex` handles a bijection $e : V \simeq V$. It moves the block to $e(s)$ and transforms the factor to $A'_{ik} = A_{i,e^{-1}(k)}$. The resulting Gram entry is unchanged. Permuting only the coordinates without moving their factor columns falls outside this statement.

3 Streamed actions and the sum-of-squares identity

`matrixAction_gram_eq_streamed` proves

$$\sum_{j\in I}\left(\sum_{k\in s} A_{ik}A_{jk}\right) v_j = \sum_{k\in s} A_{ik}\left(\sum_{j\in I} A_{jk}v_j\right).$$

This is the exact algebra behind multiplication by a sample Gram matrix without constructing that matrix. Lean checks the finite-sum interchange and associativity of real multiplication. It does not assert that a given sequence of device operations has exact real semantics.

The next theorem, `quadraticForm_gram_eq_energy`, proves

$$v^\top K_s v = \sum_{k\in s}\left(\sum_{i\in I} A_{ik}v_i\right)^2.$$

It expands the action, interchanges the finite sums, factors each coordinate sum, and identifies the product of a projection with itself. The resulting `quadraticForm_gram_nonneg` states $0 \le v^\top K_s v$ for every real vector v. This is positive semidefiniteness of the supplied Gram form.

These identities are useful together. Refinement preserves the represented operator, streaming preserves its action, and the energy identity verifies its positivity. They do not require the eigenvalues or the spectral theorem. The full engine subsequently uses numerical rank compression and spectral information, whose correctness contracts are separate.

4 A complete exact algebra example

Consider one sample coordinate and two variant columns with

$$A = (3 \quad 4), \qquad v = 2.$$

The two singleton Gram contributions are 9 and 16. Their disjoint union has Gram entry 25. Matrix multiplication gives $Kv = 50$ and $v^\top Kv = 100$. The streamed action instead computes

$$3(3\cdot 2) + 4(4\cdot 2) = 18 + 32 = 50.$$

The energy expression gives

$$(3\cdot 2)^2 + (4\cdot 2)^2 = 36 + 64 = 100.$$

Thus the engine statistic with the stated half normalization is $Q = 50$. All paths are exact real identities. This example explains the type and normalization choices; the universal theorems establish them for arbitrary finite sample coordinates and variant blocks.

If the two blocks overlap in one column, their sum repeats that column's outer product. For $s = t = \{1\}$ and $A_{i1} = 3$, the union has contribution 9 while the sum of the two blocks is 18. This is why `gramEntry_union` requires disjointness.

5 Supplied coefficient intervals

The separate definition

$$\text{weightedSquares}(s, c, z) = \sum_{k \in s} c_k z_k^2$$

models a supplied diagonal quadratic form. Its coefficients need not be eigenvalues obtained by a proved diagonalization. It has three theorems:

- `weightedSquares_mono`: if $l_k \le u_k$ for every $k \in s$, then $\sum l_k z_k^2 \le \sum u_k z_k^2$.
- `weightedSquares_nonneg`: if $c_k \ge 0$ for every $k \in s$, then $\sum c_k z_k^2 \ge 0$.
- `weightedSquares_tail_imp`: coefficientwise order gives the pointwise implication $q \le \sum l_k z_k^2 \Rightarrow q \le \sum u_k z_k^2$.

The first and third allow signed coefficients because squares remain nonnegative. The second has an explicit nonnegativity hypothesis.

The pointwise implication is a prerequisite for probabilistic tail sandwiching. It does not itself state a probability inequality or build a coupling. To turn it into such a statement, one must supply measurable random coordinates on a common probability space and use monotonicity of measure. To apply it to matrix spectra, one must also establish that each true coefficient lies inside the supplied interval.

6 Enclosures and error ledgers

The interval module uses two elementary predicates:

$$\begin{aligned} \text{Enclosed}(x, L, U) &:\iff L \le x \le U, \\ \text{ErrorBound}(x, \widehat{x}, r) &:\iff |x - \widehat{x}| \le r. \end{aligned}$$

The equivalence `errorBound_iff_enclosed` proves

$$|x - \widehat{x}| \le r \quad \iff \quad \widehat{x} - r \le x \le \widehat{x} + r.$$

There is no extra nonnegativity hypothesis: if this predicate holds, then the radius is necessarily nonnegative. The definition accepts arbitrary real inputs, and the theorem states the exact logical equivalence.

For the K-COS error ledger, distinguish the ideal analytic quantity x, an exact finite transform m, and the computed estimate $\widehat{x}$. If

$$|x - m| \le a, \qquad |m - \widehat{x}| \le b,$$

then `errorBound_compose` proves

$$|x - \widehat{x}| \le a + b.$$

The proof is the triangle inequality. The hypotheses must be proved independently. In particular, a can contain a support/alias bound and an omitted-mode bound; this theorem only composes those supplied facts.

`errorBound_accept` adds the hypothesis $a + b \le \varepsilon$ and concludes $|x - \widehat{x}| \le \varepsilon$. This is the arithmetic core of acceptance by a proved budget. A sequence of numerically stable estimates, without valid bounds, does not satisfy its hypotheses.

7 Safe transformations of an enclosure

The remaining interval statements are reusable exact-real rules. `enclosed_add` adds two enclosed quantities. `enclosed_scale` scales the interval by a nonnegative real factor. `enclosed_widen` permits an explicitly wider interval. `enclosed_intersect` combines two independently valid enclosures of the same quantity using max on the lower endpoints and min on the upper endpoints.

`enclosed_positive` requires a strictly positive lower endpoint. It then proves that the enclosed quantity is positive. This is relevant before applying a logarithm to a corrected tilted finite sum. A lower endpoint equal to zero does not establish positivity.

Finally, `enclosed_monotone` transports an interval through a supplied monotone function. Its hypothesis is global monotonicity on $\mathbb{R}$. The source does not directly instantiate this rule with the real logarithm, which needs a positive-domain formulation. A log-tail certificate therefore still needs both positivity and a separate domain-correct monotonicity argument. The interval-library implementation also requires its own correctness contract.

8 Declaration inventory and manuscript concordance

All names below are prefixed by `KomorebiQCOS`. The inventory counts project theorems only; imported mathlib theorems are not counted.

Declaration	Exact formal scope
`gramEntry_empty`	Empty variant block gives zero.
`gramEntry_singleton`	One column gives its outer product.
`gramEntry_symm`	Real Gram entry symmetry.
`gramEntry_union`	Disjoint binary block refinement.
`gramEntry_reindex`	Factor-column permutation invariance.
`matrixAction_gram_eq_streamed`	Gram action equals streamed factor action.
`matrixAction_add`	Operator-action additivity.
`matrixAction_gram_union`	Action refinement invariance.
`streamedAction_union`	Streamed accumulation over disjoint blocks.
`quadraticForm_add`	Quadratic-form additivity in the matrix.
`quadraticForm_gram_eq_energy`	Gram form equals factor energy.
`quadraticForm_gram_nonneg`	Positivity of the supplied Gram form.
`quadraticForm_gram_union`	Statistic refinement invariance.
`weightedSquares_mono`	Pointwise coefficientwise order.
`weightedSquares_nonneg`	Positivity for nonnegative coefficients.
`weightedSquares_tail_imp`	Pointwise tail-event implication.
`enclosed_order`	A valid enclosure has ordered endpoints.
`enclosed_add`	Additive interval composition.
`enclosed_scale`	Nonnegative scaling.
`enclosed_widen`	Valid interval widening.
`enclosed_intersect`	Intersection of valid enclosures.
`errorBound_iff_enclosed`	Absolute-error/enclosure equivalence.
`errorBound_compose`	Analytic and arithmetic budget composition.
`errorBound_accept`	Acceptance below a supplied tolerance.
`enclosed_positive`	Positive lower endpoint proves positivity.
`enclosed_monotone`	Supplied monotone transformation.

The exact-block, hierarchical-blocking, and streamed-action sections of the traditional paper map to the Gram theorems under the definitions above. The error-ledger and acceptance sections map only to the

arithmetic consequences of already valid bounds. The matrix-to-tail bridge maps to pointwise coefficient order only. The detailed machine-readable concordance includes source filenames and implementation anchors.

9 Reproduction and trusted dependencies

The project includes `lean-toolchain`, `lakefile.toml`, the pinned manifest, two mathematical modules, a root import, an axiom-audit file, controls, and fresh build logs. On a normal Lean installation, run the following from the companion directory:

```
lake update
lake exe cache get
lake build
lake env lean AxiomAudit.lean
```

The dependency revision in the project configuration is exact. The large compiler and mathlib cache are omitted from the publication ZIP. Reproduction requires the pinned versions and an appropriate cache or a dependency build.

The producing environment needed a small executable-self-location adapter. Its source and explanation are included with the runtime evidence. The compiler and proof kernel were not edited. Positive and negative controls check that the compiler accepts a valid arithmetic proposition and rejects a false one with an invalid proof; these controls supplement the theorem build.

Lean's axiom audit reports the dependencies of each project theorem. Classical choice, propositional extensionality, and quotient soundness are normal foundational dependencies in this mathlib development. The audit must not contain `sorryAx`, which would indicate an admitted proof. An empty admission scan alone would not establish acceptance: the fresh compiler build and axiom output supply that evidence.

10 Remaining formal obligations

Extending this companion toward an end-to-end certificate would require the following mathematical and implementation layers:

1. Gaussian whitening and central, signed, and noncentral quadratic-form reduction under their exact hypotheses.
2. Finite-dimensional spectral decomposition and a proved link between the input matrix, its factor, and supplied coefficient intervals.
3. Probability monotonicity under a common measurable coupling.
4. Exponential tilting and the transform/payoff inversion identity.
5. Support/alias and omitted-mode remainder bounds, including rank and moment-generating-function-domain conditions.
6. Exact interval semantics for decimal inputs, directed rounding, roots, complex transforms, logarithms, and transcendental library operations.
7. A verified bridge from the numerical implementation and its error ledger to those mathematical bounds, including input parsing and output endpoint conversion.

These obligations are distinct. Formalizing the finite algebra does not silently discharge any analytic or numerical layer. The present companion is complete for the finite-algebra and arithmetic claims listed in its inventory, with the recorded compiler and axiom evidence.